澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

Virtual Memory: Operating System Support

Memory Management Algorithms & OS Policies

Today's Roadmap: Four Core OS Policies

PART 01

Fetch Policy

When to bring pages into memory: Demand paging vs. Prepaging

PART 02

Replacement Policy

Which page to evict: Optimal, LRU, FIFO, Clock algorithms

PART 03

Resident Set Management

How many frames per process: Fixed vs. Variable allocation

PART 04

Frame Locking & Scope

Which frames can be replaced: Locked frames, local vs. global scope

---

👉 Core question: How does the OS manage limited physical memory across multiple processes?

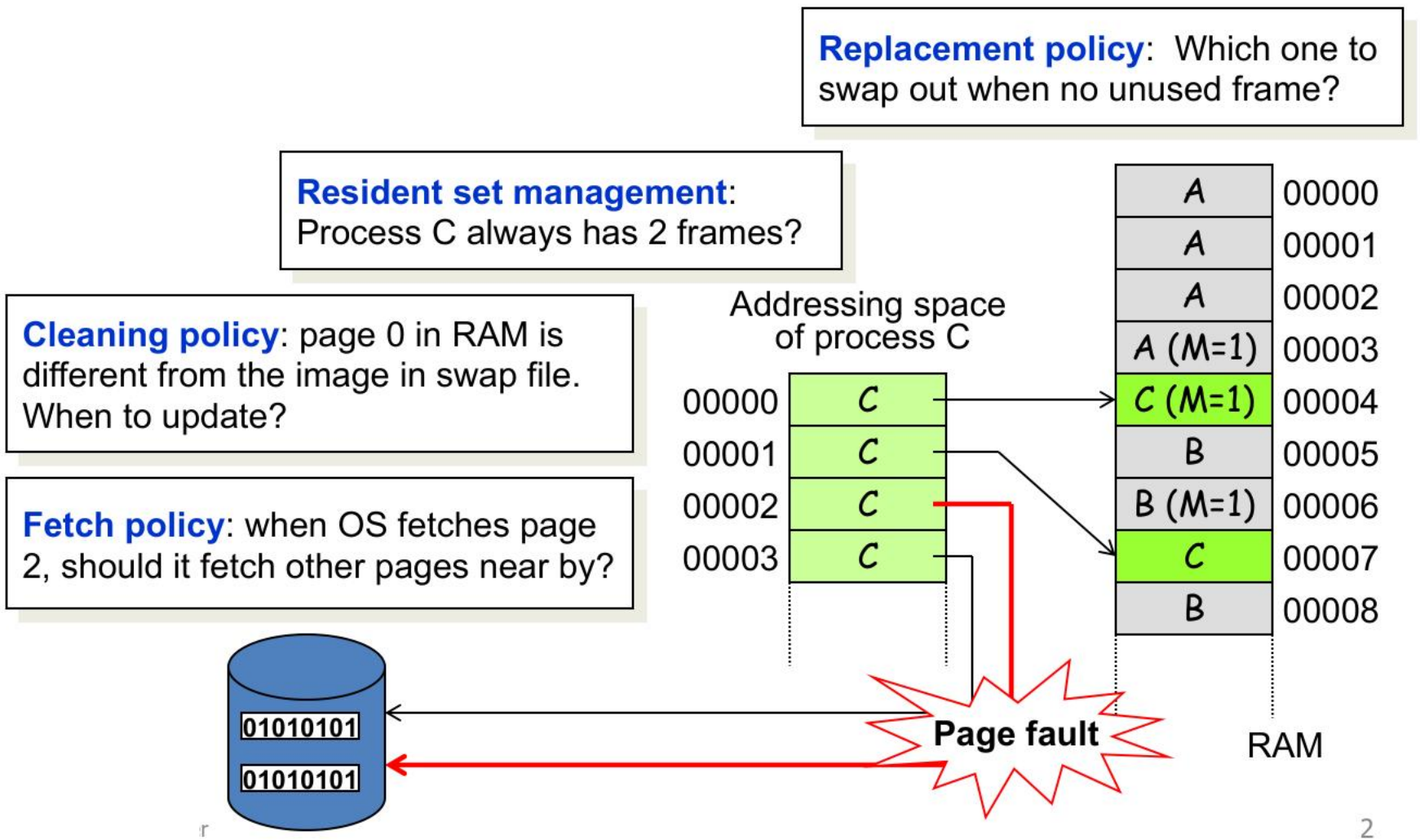
OS Policies for Virtual Memory

Replacement policy: Which one to swap out when no unused frame?

Resident set management: Process C always has 2 frames?

Cleaning policy: page 0 in RAM is different from the image in swap file. When to update?

Fetch policy: when OS fetches page 2, should it fetch other pages near by?



Frame Locking

- Some frames in main memory may be locked, and cannot be replaced.
- Examples of locked frames:
  - much of the OS kernel (e.g. process management and memory management)
  - key control structures (e.g. ready queue)
  - I/O buffer (chap 11)

Resident Set

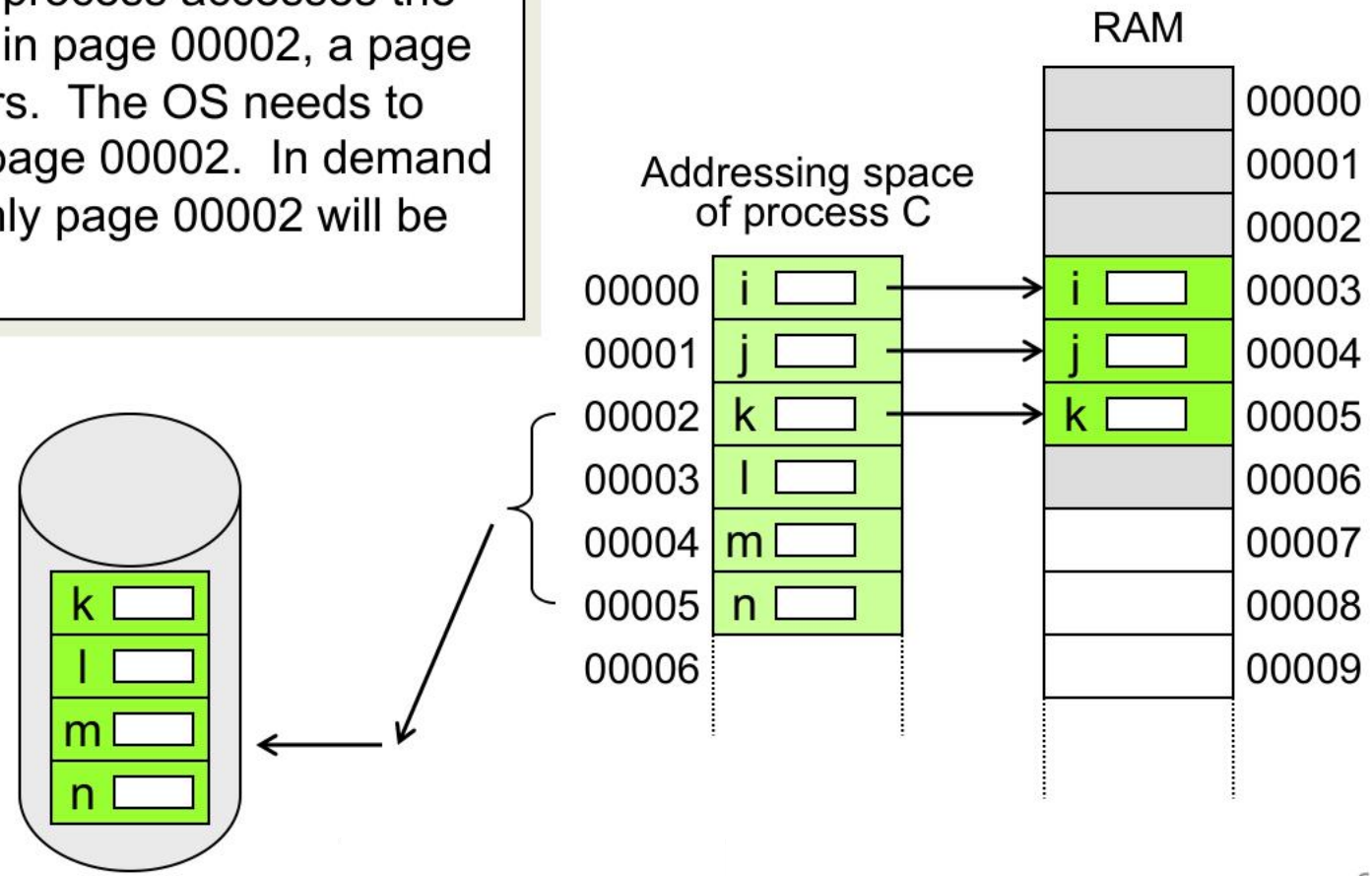
- Resident set – portion of process that is in main memory
- Resident set size – number of frames allocated to a process

Fetch Policy

- determines when a page should be brought into memory
  - demand paging – only brings pages into main memory when a reference is made to a location on the page
    - many page faults when process first started
  - prepaging – brings in more pages than needed
    - more efficient to bring in pages that reside contiguously on the disk

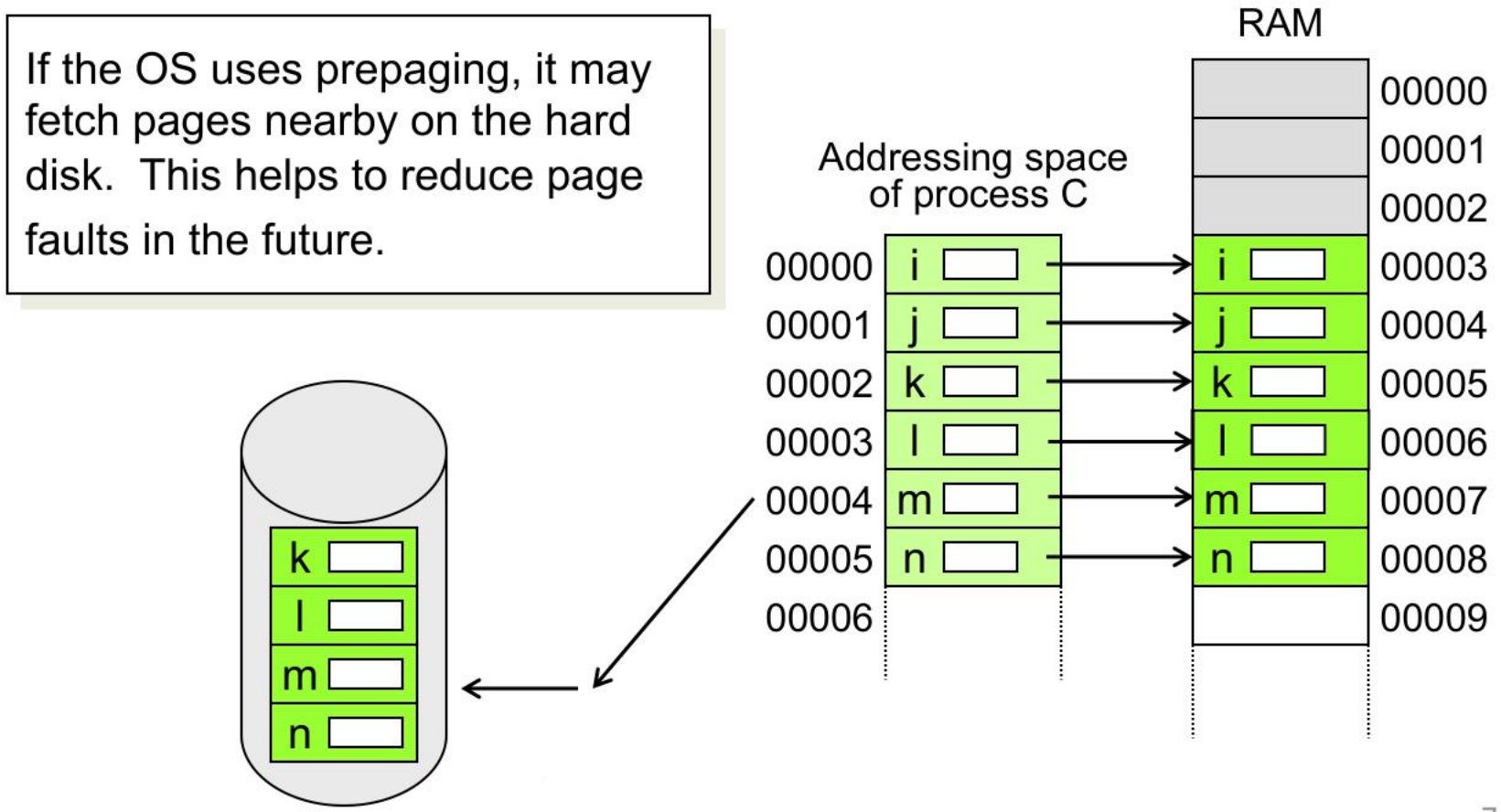
Fetch Policy – Demand paging

When the process accesses the variable k in page 00002, a page fault occurs. The OS needs to fetch the page 00002. In demand paging, only page 00002 will be fetched.



Fetch Policy – Prepaging

If the OS uses prepaging, it may fetch pages nearby on the hard disk. This helps to reduce page faults in the future.



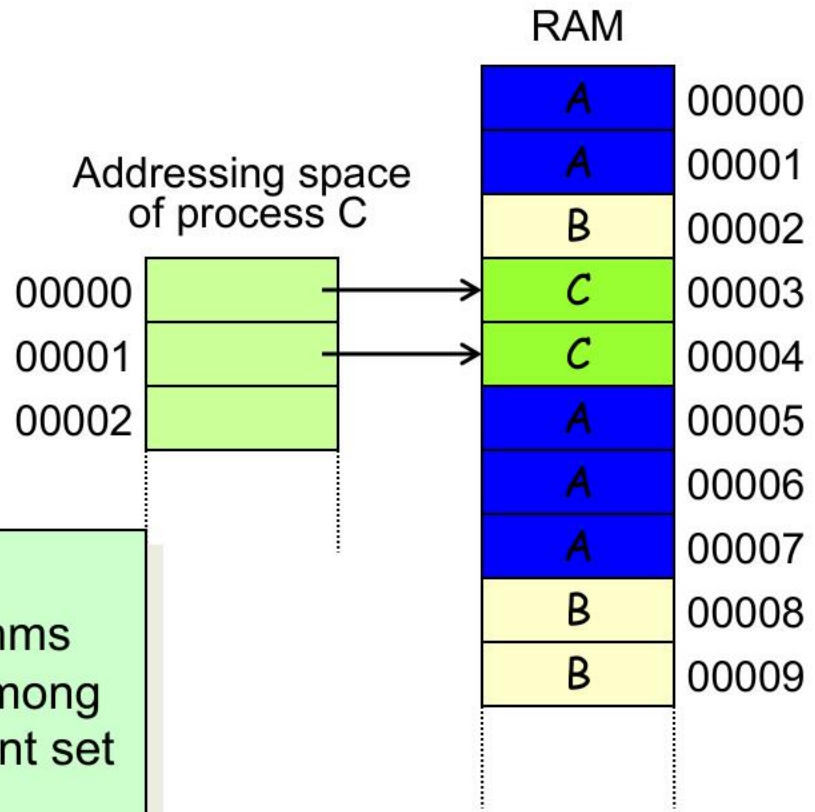
Replacement Policy

- Selection of a page in memory to be replaced when a new page is brought in
  - locked frames should not be replaced
- Two related questions:
  - whether the set of pages to be considered for replacement should be limited to those of the process that causes the page fault?
  - how many frames are to be allocated to each process?
  - These are considered in resident set management

Replacement scope

Process C causes a page fault when it references data in page 00002. And there is no unused frames in RAM. The OS has to select a frame to replace. It can choose from

- frames allocated to process C: 00003 and 00004, or
- all frames



We isolate this issue in the resident set management. The replacement algorithms we will discuss just choose one page among the pages limited by the policy of resident set management.

Replacement Policy

- We'll study four replacement policies:
  - Optimal
  - LRU
  - FIFO
  - Clock

Comparison

	Performance	… to implement
Optimal	Optimal	Impossible
LRU	Near optimal	Expensive
FIFO	Not good	Simple
Clock	Good	OK

Replacement Policy – Optimal

- Optimal policy
  - Replaces the page for which the time to the next reference is the longest
  - Results in the fewest number of page faults
  - Impossible to implement – impossible to have perfect knowledge of future events

Replacement Policy – LRU

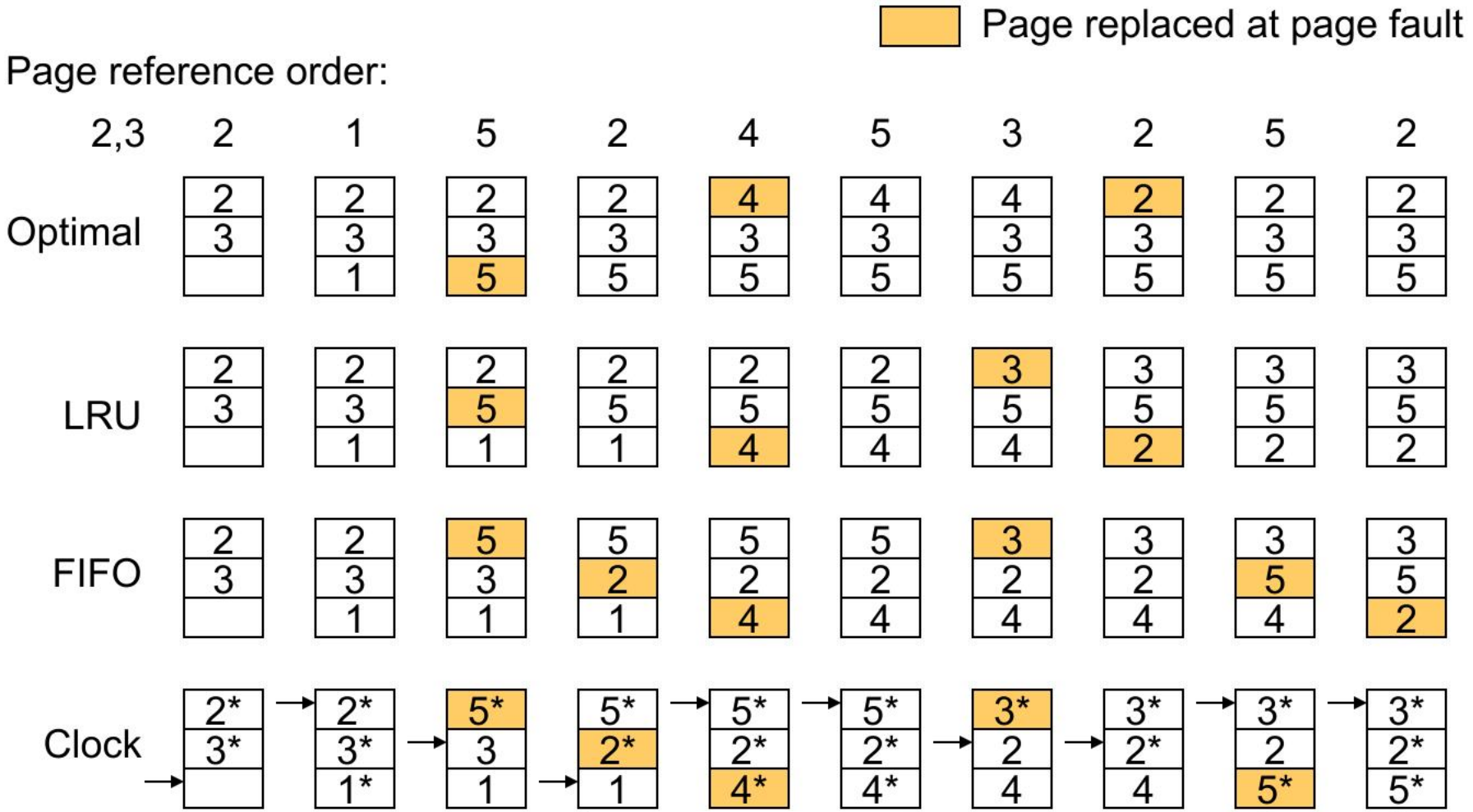
- Least Recently Used (LRU)
  - Replaces the page that has not been referenced for the longest time
  - By the principle of locality, this should be the page least likely to be referenced in the near future
  - Performs nearly as well as Optimal
  - Difficult to implement – each page would be tagged with the time of last reference

Replacement Policy – FIFO

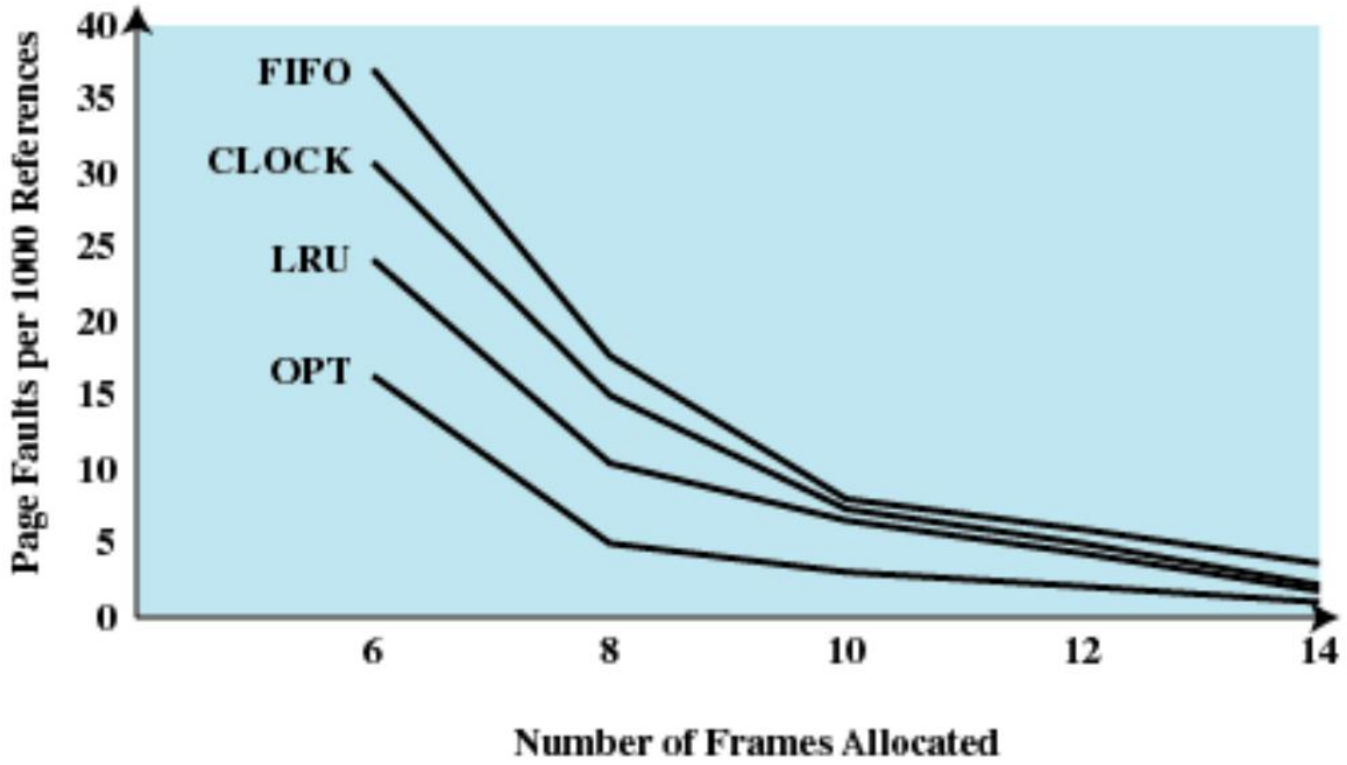
- First-in, first-out (FIFO)
  - Replaces the page that has been in memory the longest
  - Simple to implement
  - Not good performance – replaced pages may be needed again very soon

Read Fig 8.15

Replacement Algorithms



Comparison of Page Fault Rate of Replacement Algorithms

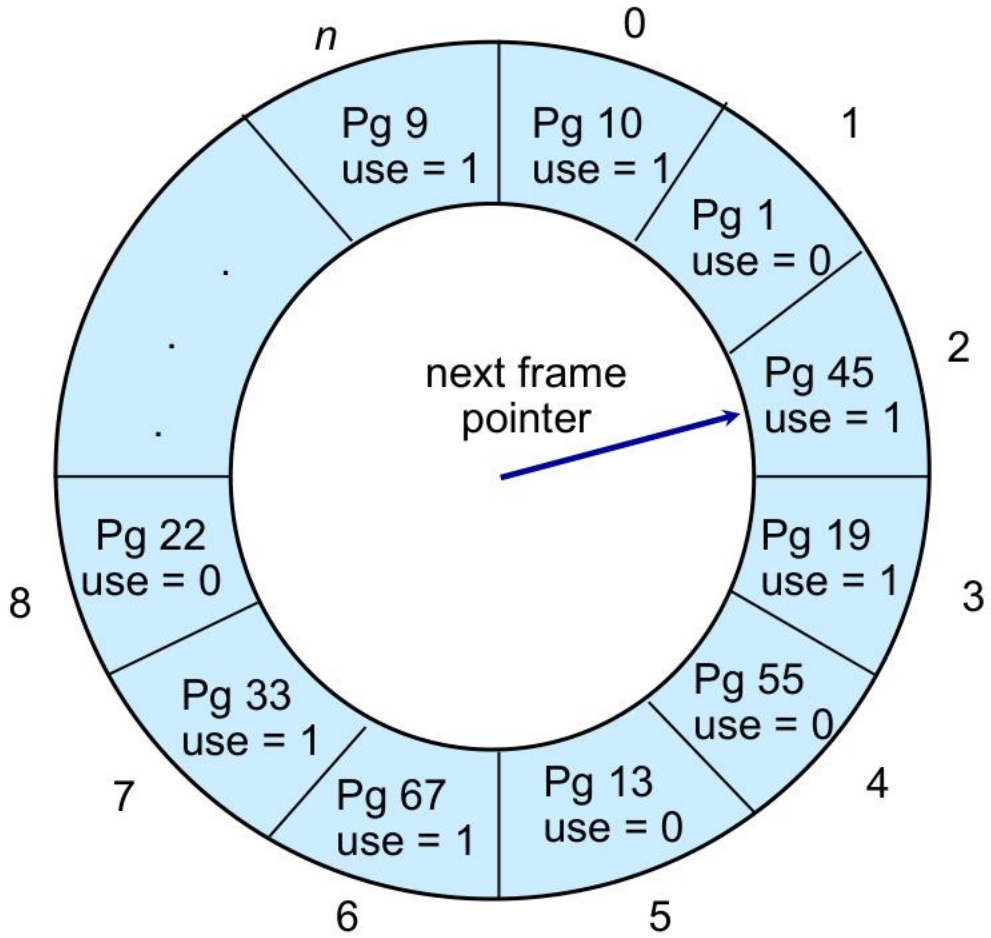


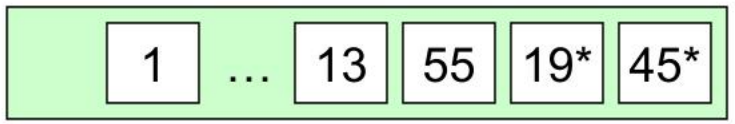
Replacement Policy – Clock

- Clock Policy
  - Needs an additional bit U (Use bit) in PTE
    - when a page is first loaded in memory, OS sets U=1
    - when the page is referenced, CPU sets U=1
  - Works like FIFO
    - the first frame encountered with U=0 is replaced
    - during the search for replacement, OS resets U= 0

Hence, U bit represents whether the page is ‘recently’ used

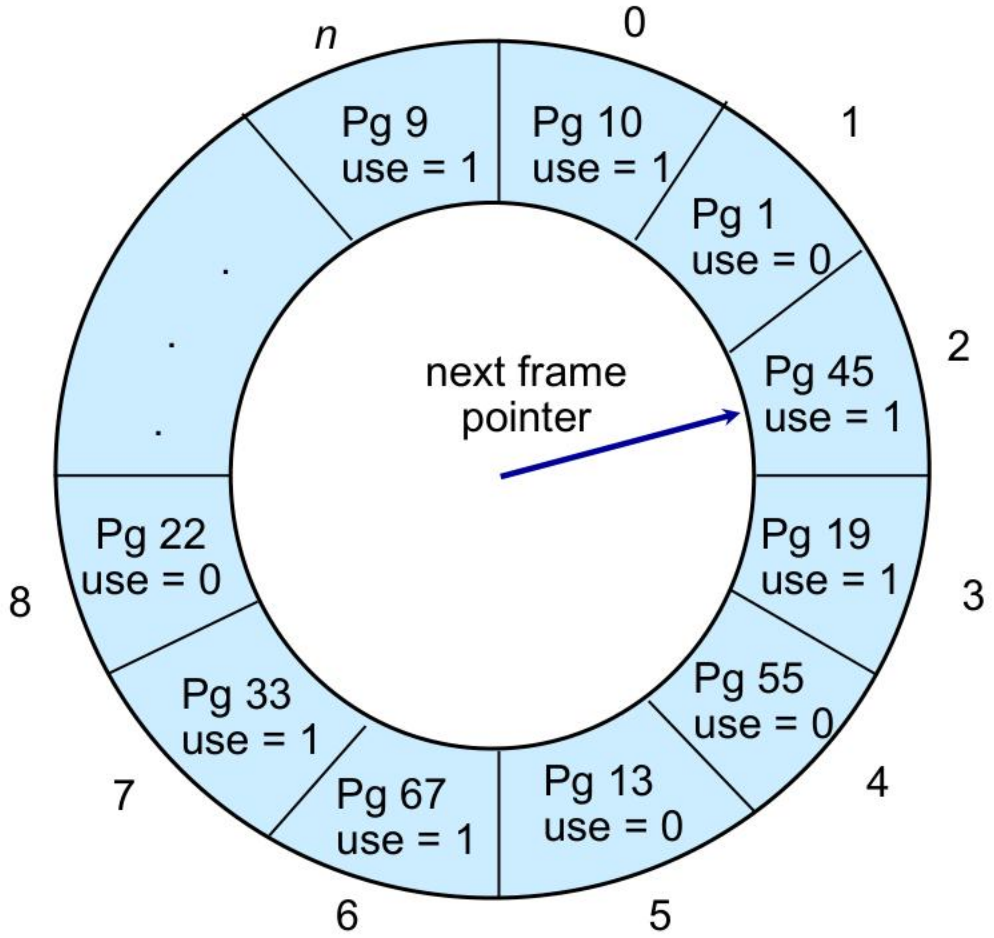
Clock Policy, 1

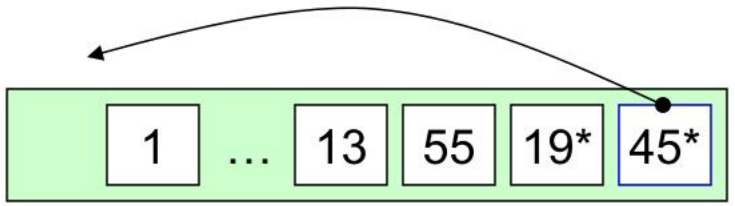




At this moment, the running process tries to access page 77, which is not present. A page fault occurs.

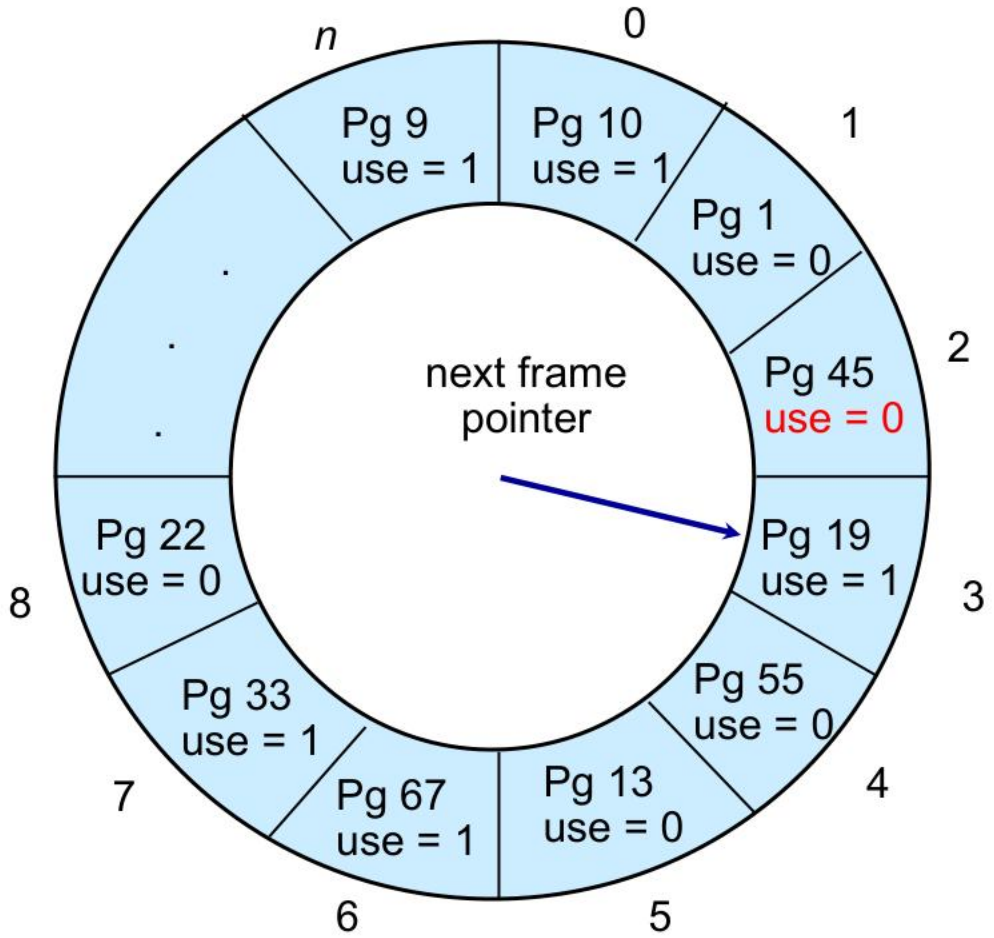
Clock Policy, 2

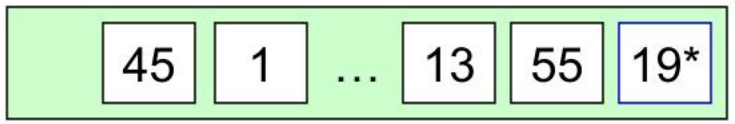




Page 45 has been loaded into memory the longest. In FIFO, we would replace it although it has been used recently (U=1). However, in clock, we give it a second chance and put in at the end of the queue again.

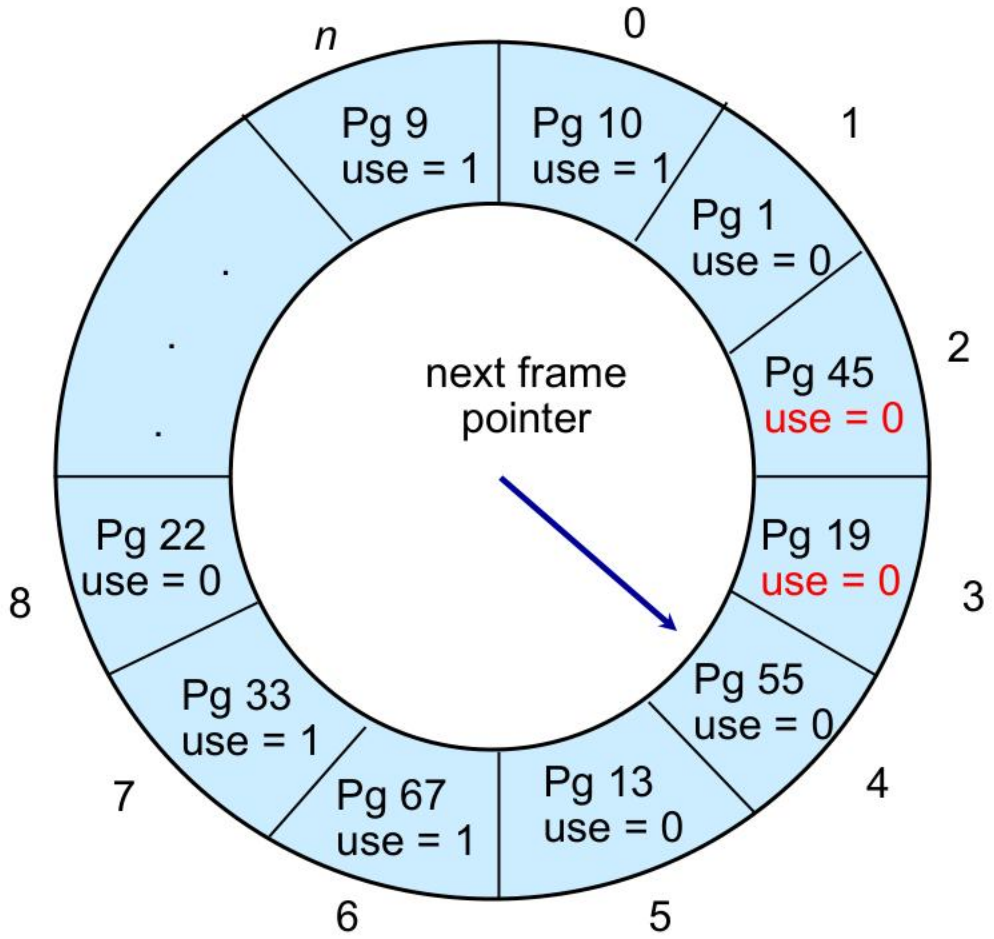
Clock Policy, 3





Similarly, page 19 has U=1 and therefore it will not be replaced yet. The OS goes on to check the next oldest page.

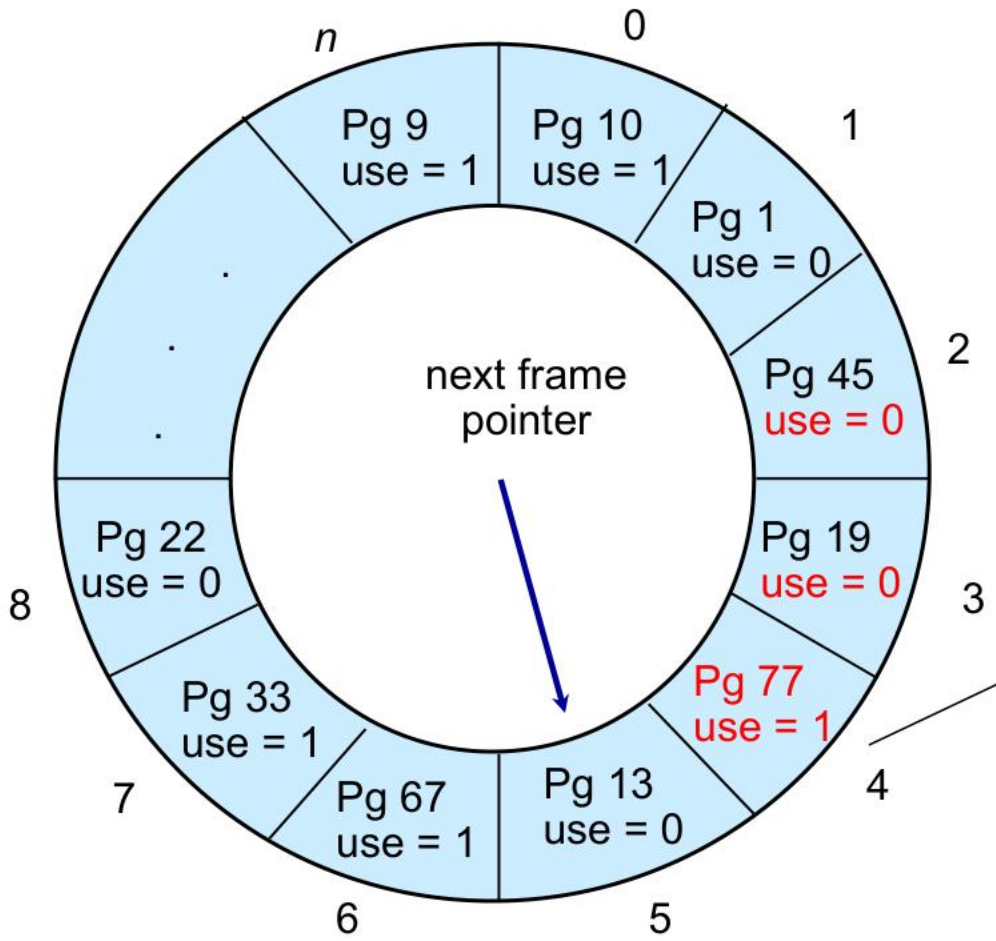
Clock Policy, 4





Page 55 is the 'oldest' page in RAM, and is not used recently. So the OS replaces this page with page 77.

Clock Policy, 5

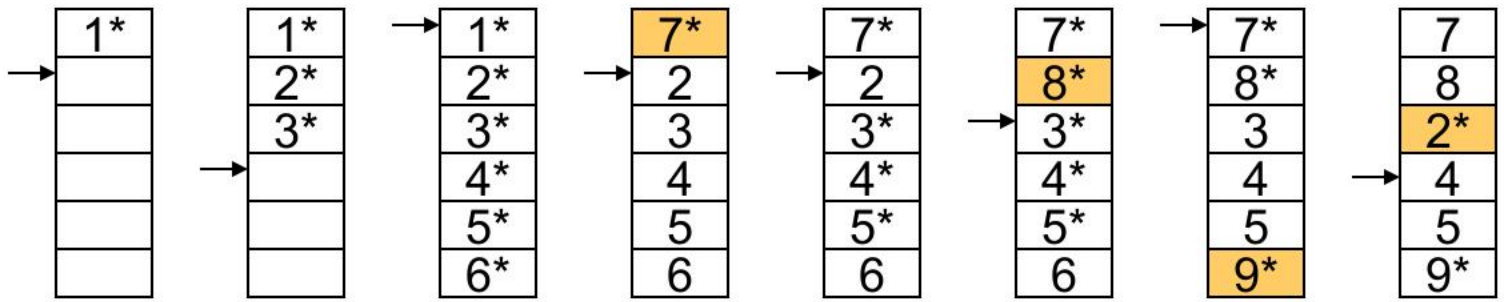




Page 77 replaces page 55. It is put at the end of the queue (it is the youngest page in RAM). In next page fault, the OS will start the searching from page 13, the oldest page.

Clock Policy – another example

Page reference order: 1 | 2,3,2 | 4,5,6 | 7 | 4,3,5,3 | 8 | 9,8 | 2

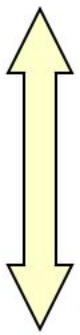


The 'next frame pointer' sweeps the whole buffer once. This resets the U bit of all pages to 0. When the pointer hits the first entry of the buffer again, the page 1 is swapped out.

When the pages 4,3,5,3 are referenced, their U bit is set to 1. Notice the pointer does NOT move.

Categories of frames using U, M

U=0, M=0	Not accessed recently, not modified
U=0, M=1	Not accessed recently, modified
U=1, M=0	Accessed recently, not modified
U=1, M=1	Accessed recently, modified



Most favorable to replace

Should be expensive to replace

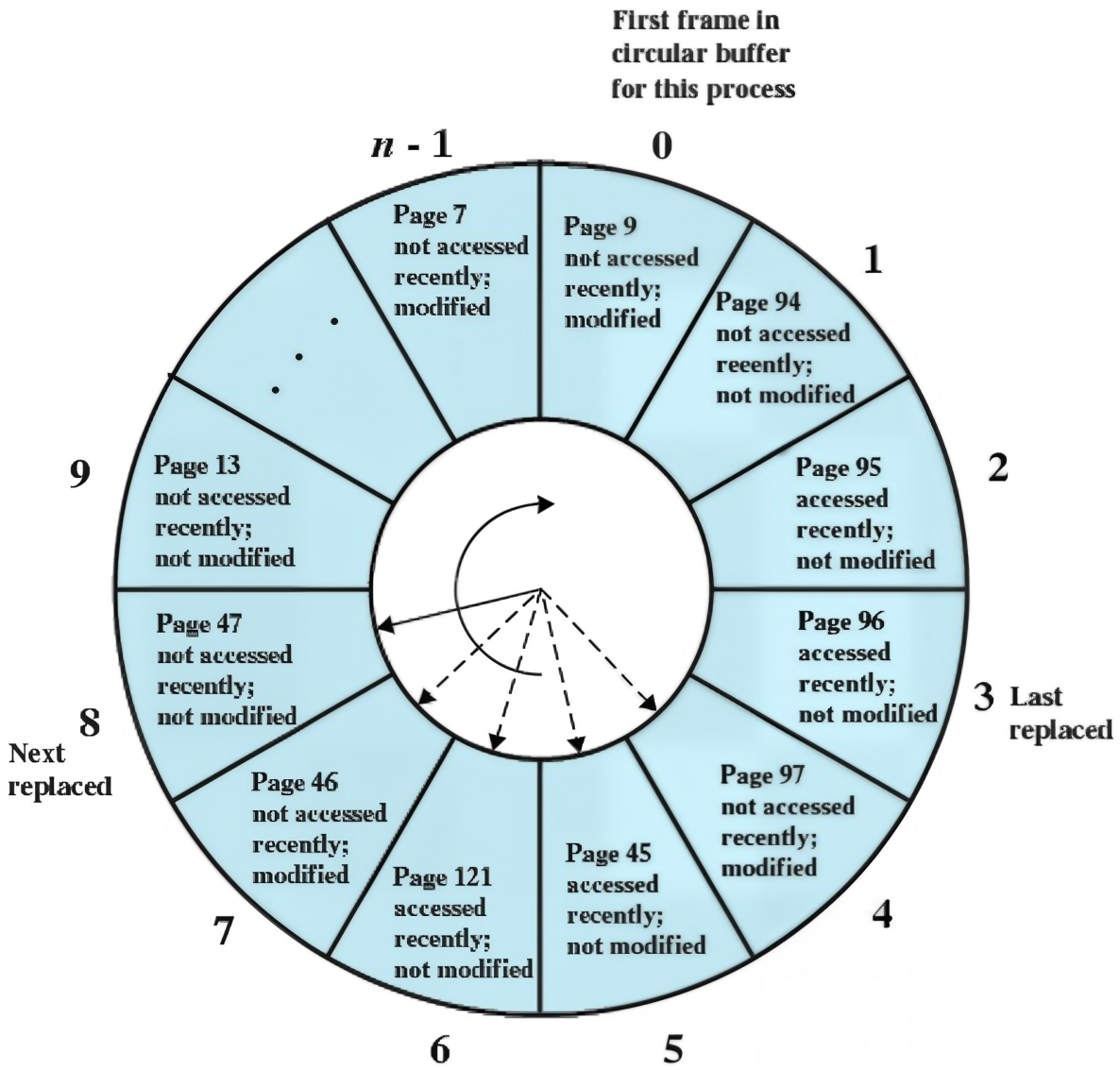
Clock using U and M bits

1. Beginning at the current position of the pointer, scan the frame buffer. During this scan, make no changes to the U bit. The first frame encountered with U=0 M=0 is selected for replacement.

2. If step 1 fails, scan again, changing U=0 on each frame bypassed. Look for frame with U=0 and M=1 to replace.

3. If step 2 fails, the pointer should have returned to its original position and all the frames in the set will have U=0. Repeat step 1 and, if necessary, step 2. This time, a frame will be found for the replacement.

Clock using U and M bits



Resident set

- Resident set – portion of process that is in main memory
  - Fixed-allocation: gives a process a fixed number of pages within which to execute
  - Variable-allocation: number of pages allocated to a process varies over the lifetime of the process

Replacement scope

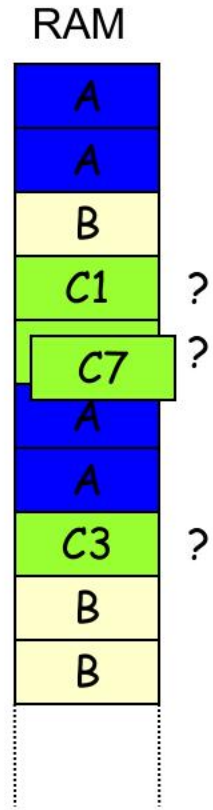
- Local scope – choose only among the resident set of the process that generated the page fault
- Global scope – choose among all unlocked pages

Resident Set Management

- Fixed allocation, local scope
- Variable allocation, global scope
- Variable allocation, local scope

Fixed allocation, Local scope

- Each process is allocated a fixed number of frames within which to execute
- At page fault, the OS replaces a page from the resident set of the process
- If the allocation is too small, there will be a high page fault rate
- If the allocation is too large, there will be too few programs in main memory

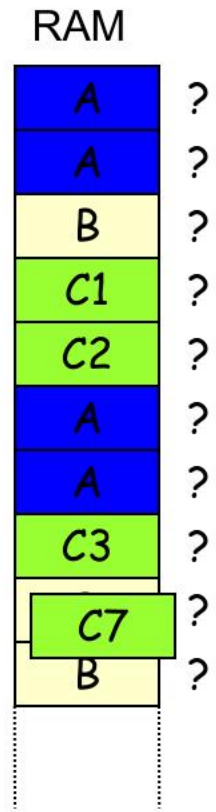


Variable allocation, Global scope

- The resident set size of each process can change at run-time
- The OS keeps a list of free frames
- Free frame is added to resident set of process when a page fault occurs
- If no free frame, replaces one from another process

Variable allocation, Global scope

- At page fault, the OS may choose any unlocked page for replacement
- An ‘active’ process gets more and more frames allocated

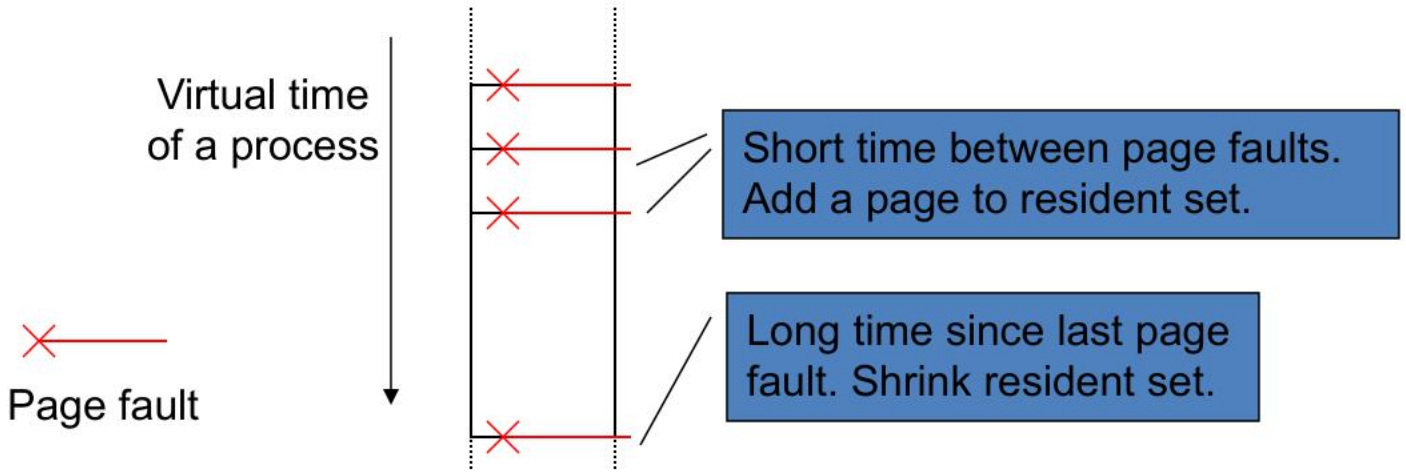


Variable allocation, Local scope

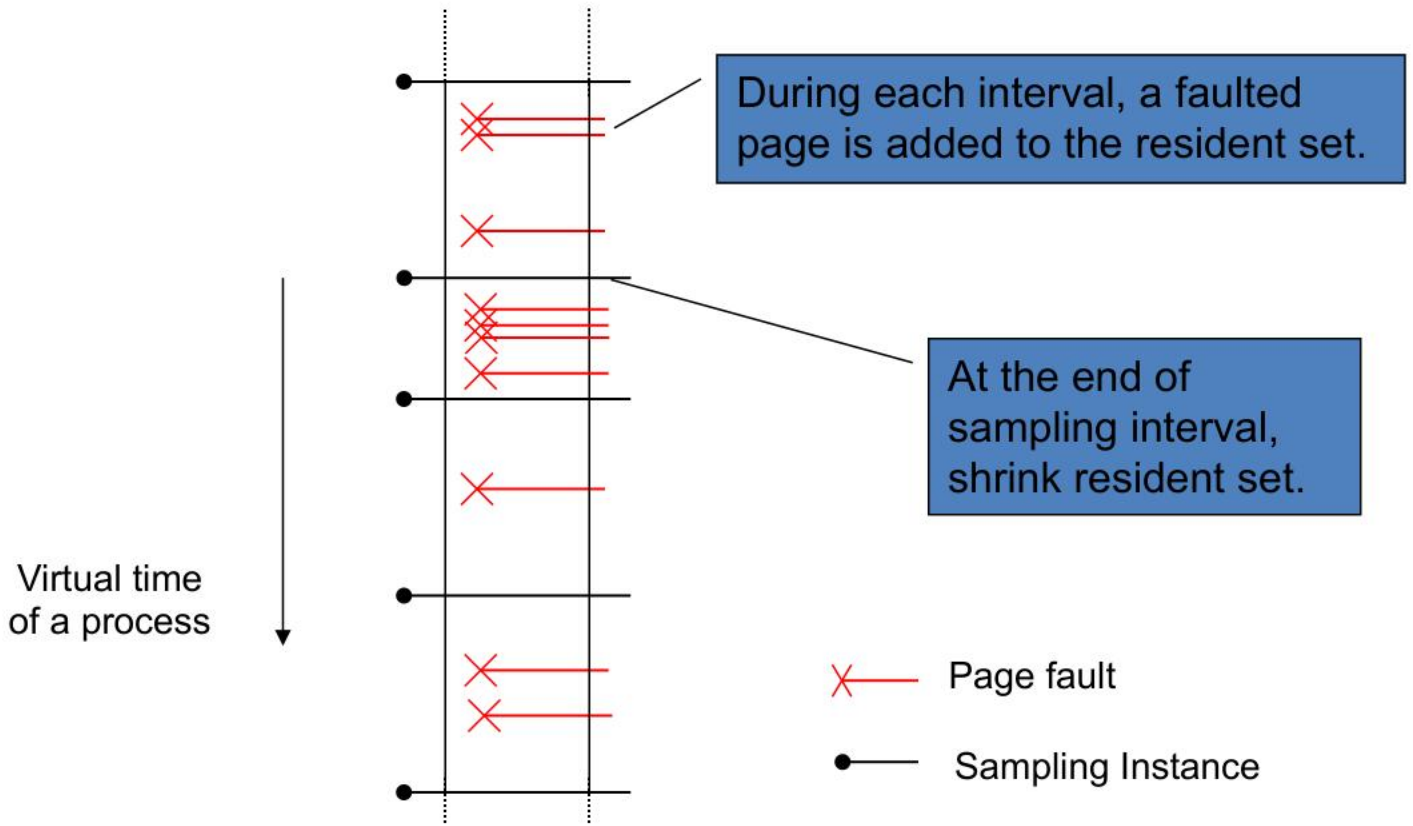
- At page fault, the OS replaces a page from the resident set of the process
- Occasionally, the OS will adjust the resident set size of the process according to its behavior
  - Page-fault frequency (PFF)
  - Variable-interval sampled working set (VSWS)

Page-fault frequency (PFF)

- When a page fault occurs,
  - if the time since last page fault is short, add a page
  - if the time since last page fault is long, discard all pages with Use bit = 0, and reset the Use bit of the rest pages.
  - otherwise, choose a page from the resident set for replacement.



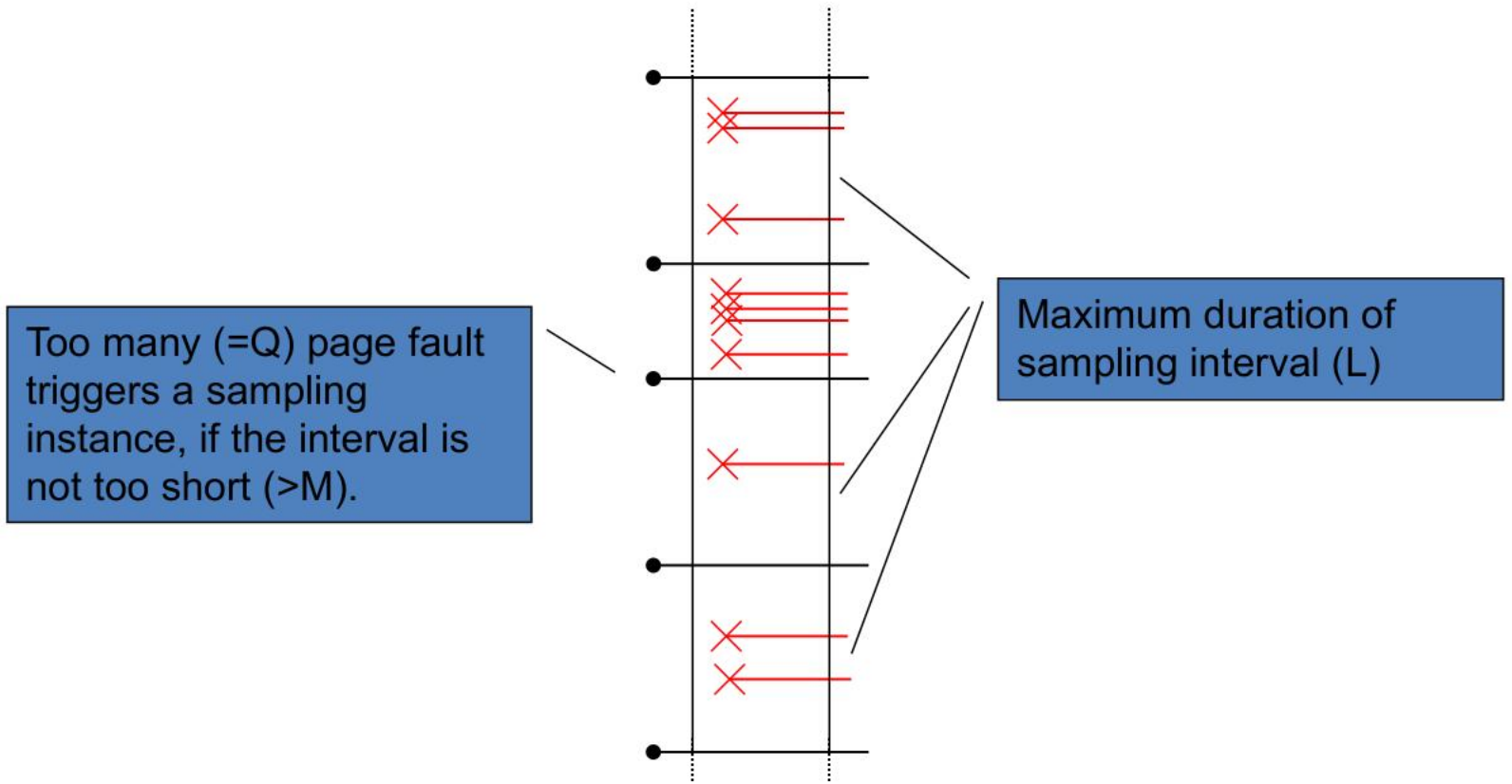
Variable-interval Sampled Working Set (VSWS)



VSWS

- At sampling instance (the end of a sampling interval)
  - Suspend the process
  - Scan the Use bit of all pages of the process
    - If U=0, discard them
    - If U=1, retain the page in the resident set. Reset U=0.
  - Resident set may shrink
- During the sampling interval
  - Any faulted pages are added to the resident set
  - Resident set may grow

VSWS


Too many (=Q) page fault triggers a sampling instance, if the interval is not too short (>M).
Maximum duration of sampling interval (L)

VSWS

- Three parameters:
  - M: min duration of sampling interval
  - L: max duration of sampling interval
  - Q: the number of page faults that are allowed to occur between sampling instances

VSWS Policy

- If the virtual time (VT) since the last sampling instance reaches L, then suspend the process and scan the use bits
- If, prior to an elapsed VT of L, Q page faults occur,
  - If the VT since the last sampling instance < M, then wait until the elapsed VT reaches M (to suspend the pocess and scan the use bits).
  - Or, if the virtual time since the last sampling instance is >= M, suspend the process and scan the use bits.

Cleaning Policy

- Demand cleaning
  - a page is written out only when it has been selected for replacement
- Precleaning
  - pages are written out in batches

Summary:
Usage of P, M, U bits

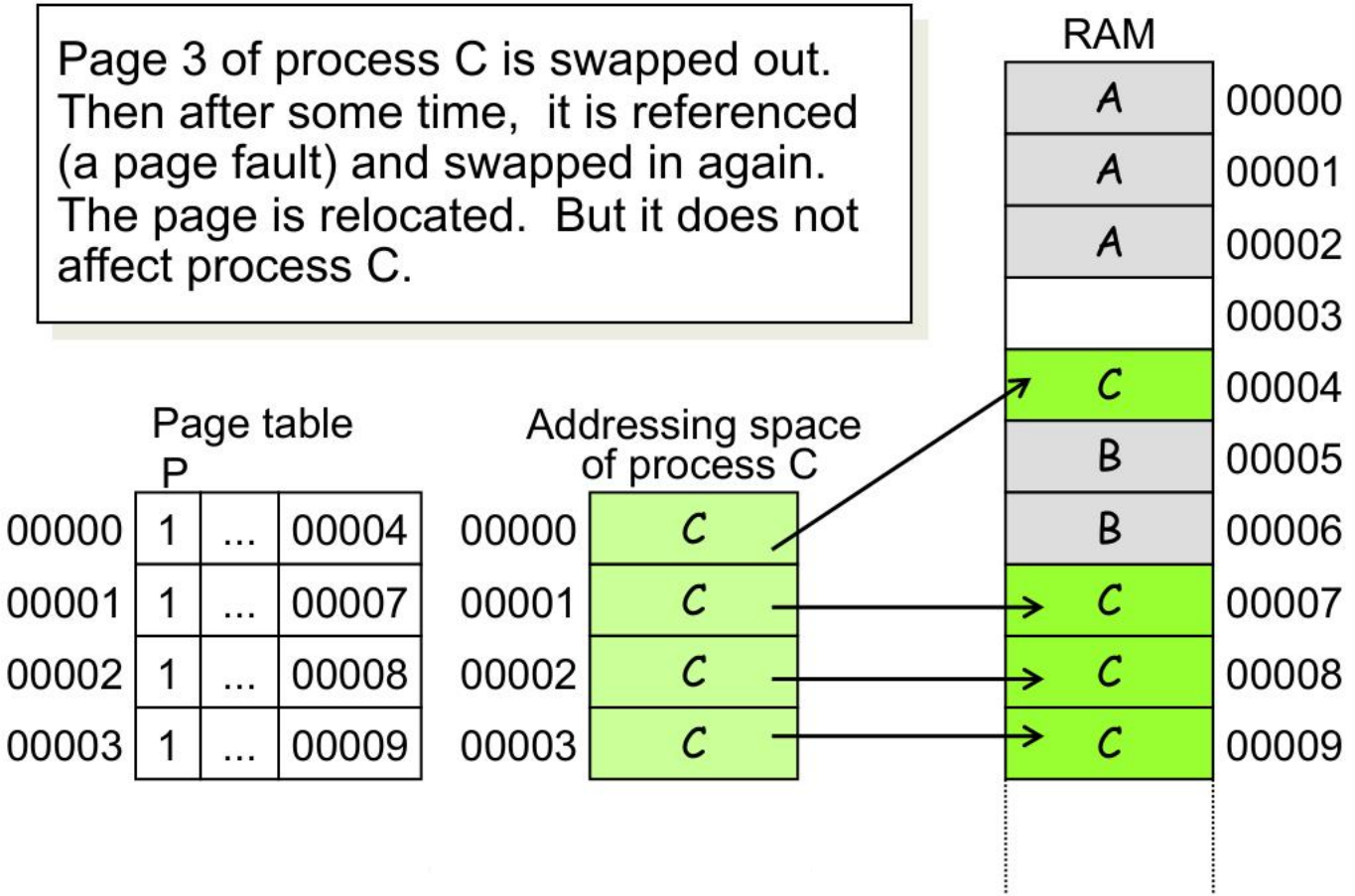
	CPU ...	OS ...
P	Checks at each memory reference: if P=0, generate page fault	Set P=0 when swap a page out. Set P=1 when swap a page in.
M	Set M=1 when any byte in the page is modified	Set M=0 when swap a page in. When swap a page out, write the page to swap file if M=1. Used in Clock.
U	Set U=1 when any byte in the page is accessed (read/write)	Used in Clock, PFF, and VSWS to determine whether a page has been referenced recently. May reset U=0.

Memory Management Requirements

- As a conclusion, we'll check how paging satisfies the requirements of memory management listed in Chapter 7
  - Relocation
  - Protection
  - Sharing

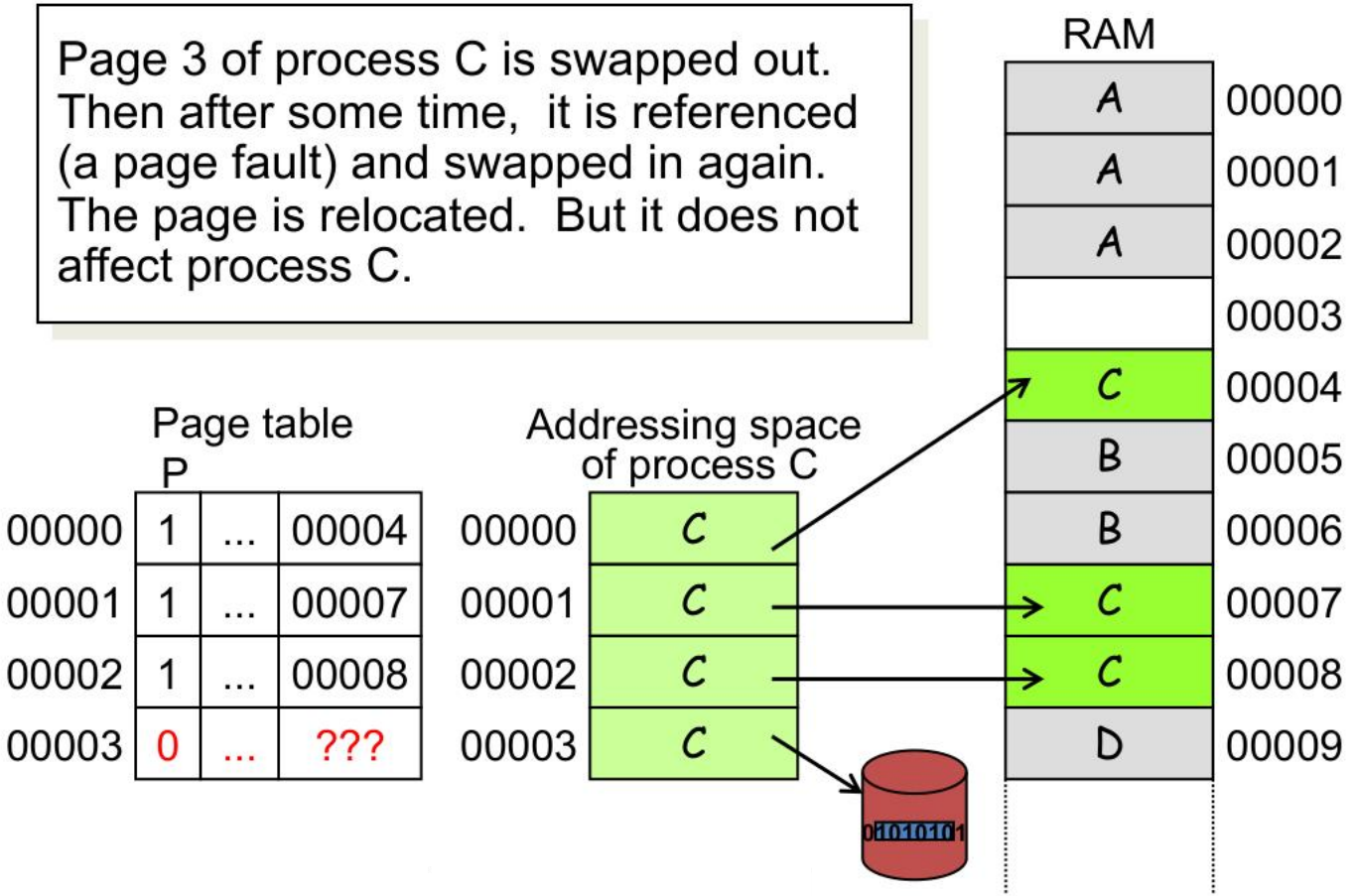
Relocation

Page 3 of process C is swapped out. Then after some time, it is referenced (a page fault) and swapped in again. The page is relocated. But it does not affect process C.



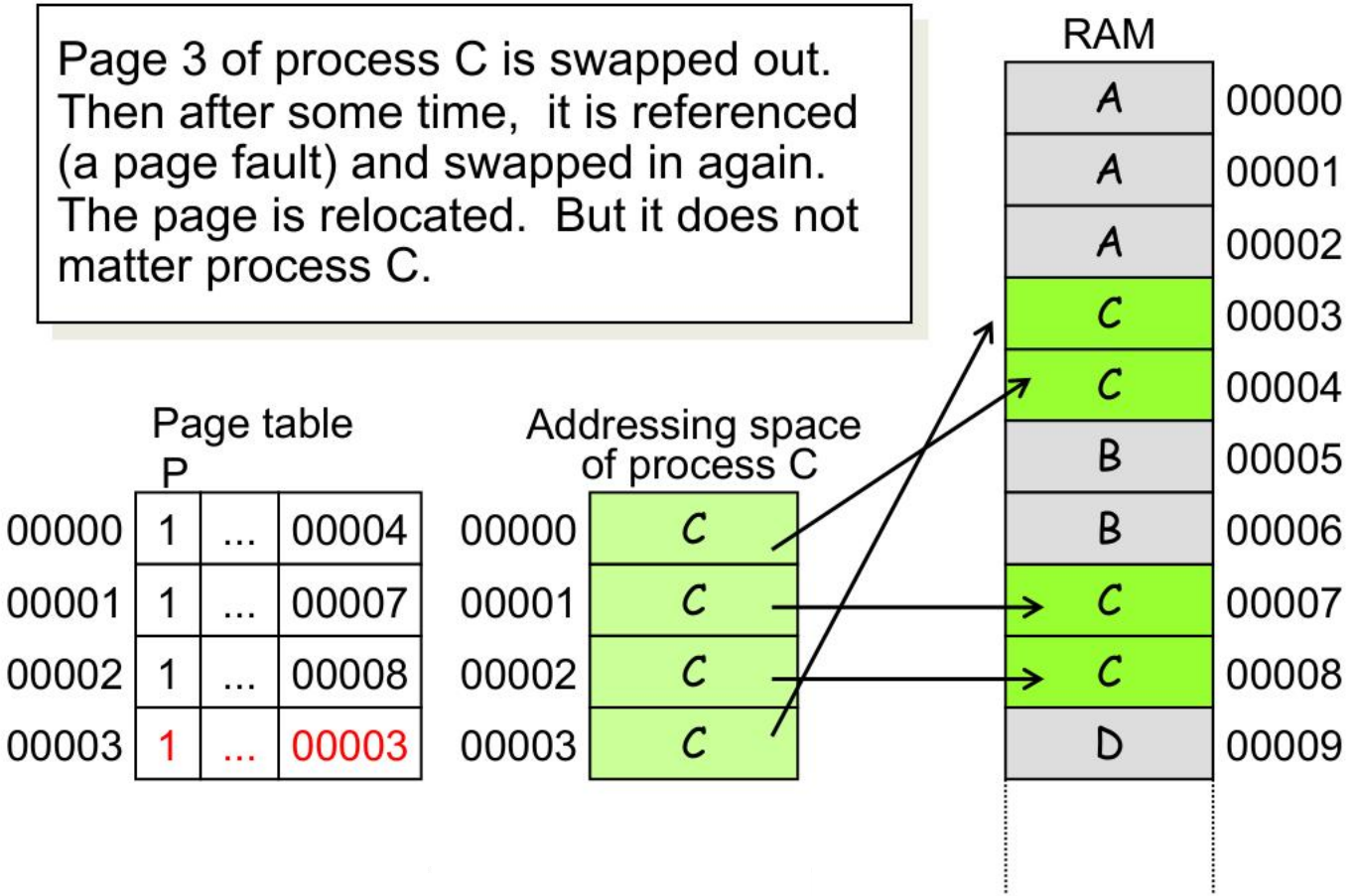
Relocation

Page 3 of process C is swapped out. Then after some time, it is referenced (a page fault) and swapped in again. The page is relocated. But it does not affect process C.



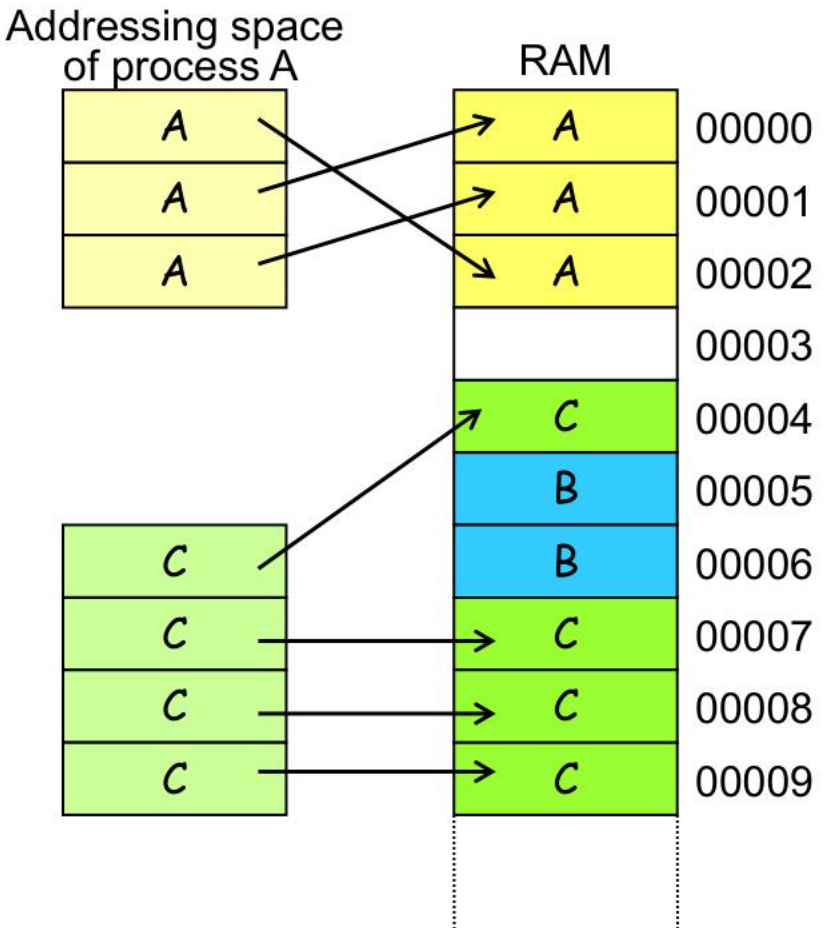
Relocation

Page 3 of process C is swapped out. Then after some time, it is referenced (a page fault) and swapped in again. The page is relocated. But it does not matter process C.



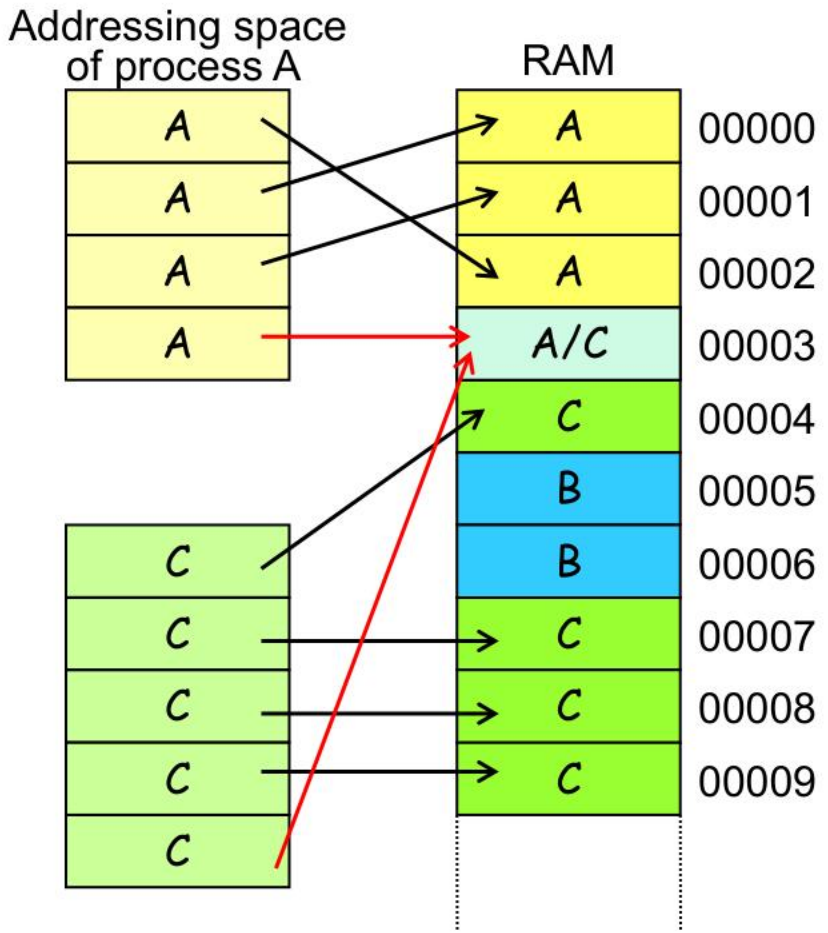
Protection

Process A has no way to access the addressing space of process C. Every memory location seen by process A is restricted within its addressing space.



Sharing

But if they agree (and the OS supports), they can share memory by mapping their own pages to the same frame.



Chapter Summary: Virtual Memory OS Support

Four core policies working together to manage limited physical memory:

Fetch Policy
Demand vs. Prepaging
When to bring pages in

Replacement Policy
OPT, LRU, FIFO, Clock
Which page to evict

Resident Set Mgmt
Fixed vs. Variable
How many frames per process

Scope
Local vs. Global
Whose pages to evict

Key Takeaway: No single policy is universally best. Real operating systems (Linux, Windows) use **Clock-based algorithms with U/M bits** as a practical compromise — close to LRU performance at low implementation cost. The choice between fixed/variable allocation and local/global scope further tunes the system for specific workloads.

豆包工作 AI 生成